

24CSI024-DATA STRUCTURES AND ALGORITHMS
UNIT 1-INTRODUCTION TO DATA STRUCTURES AND ALGORITHMS
Arrays and Array-Based Techniques (Two-Pointer, Sliding Window, Prefix Sums)
LAB WORKBOOK-16.07.2026

CATEGORY: EASY

Question 1: Merging Two Sensor Logs (Two-Pointer)

Two weather stations, A and B, each recorded temperature readings independently and stored them in non-decreasing sorted order. Divya needs to merge both logs into a single sorted array for her analysis dashboard — without using any built-in sort function, and in $O(N + M)$ time.

She uses the two-pointer merge technique:

1. Initialize a pointer i at index 0 of array A and a pointer j at index 0 of array B.
 2. Compare $A[i]$ and $B[j]$. Copy the smaller value into the output array and advance that pointer.
 3. Repeat until one array is exhausted.
 4. Copy any remaining elements of the other array directly into the output.
- The first line of input consists of two space-separated integers N and M — the sizes of arrays A and B.
 - The second line consists of N space-separated integers (array A, sorted).
 - The third line consists of M space-separated integers (array B, sorted).
 - Print a single line of $N + M$ space-separated integers representing the merged sorted array.
 - $1 \leq N, M \leq 10000$
 - $-10^4 \leq arr[i] \leq 10^4$

Input 1:

Input 2:

0 3 10	2 2 0 0 -1 4
-----------	--------------------

Output:

Output:

3 5 8 9 10	-1 0 0 4
------------	----------

Question 2: Locating the First Below-Threshold Reading (Linear Search)

Farhan monitors battery voltage readings from a fleet of IoT sensors, logged in

the order they were received (not sorted). He needs to find the position of the first reading that drops below a safe voltage threshold T , so maintenance can pinpoint when the fault began.

- The first line of input consists of two space-separated integers N and T .
- The second line consists of N space-separated integers representing the voltage readings, in the order recorded.
- Print a single integer: the 0-indexed position of the first reading strictly less than T . If no such reading exists, print -1 .
- $1 \leq N \leq 10000$
- $10^3 \leq \text{arr}[i], T \leq 10^3$

Input 1:

Input 2:

6 5	4 0
9 7 6 4 8 3	1 2 3 4

Output:

Output:

3	-1
---	----

CATEGORY: MEDIUM

Question 3: Maximum Rainwater Between Towers (Two-Pointer)

A telecom company has N transmission towers of varying heights standing in a row. During heavy rain, water can collect in the “well” formed between two towers, bounded by the shorter of the two. Meera wants to find the two towers that can trap the maximum volume of water between them, so she can plan drainage.

She solves it in $O(N)$ using two pointers instead of checking every pair:

1. Initialize $\text{left} = 0$ and $\text{right} = N - 1$, and $\text{maxWater} = 0$.
 2. Compute the water trapped between left and right : $\min(\text{height}[\text{left}], \text{height}[\text{right}]) * (\text{right} - \text{left})$. Update maxWater if this is larger.
 3. Move the pointer at the shorter tower inward — it can never form a better pair with anything closer than the taller side already offers.
 4. Repeat until left meets right .
- The first line of input consists of an integer N — the number of towers.
 - The second line consists of N space-separated non-negative integers representing tower heights.
 - Print a single integer representing the maximum water that can be trapped, in the

format: Max Water: X.

- $2 \leq N \leq 10000$
- $0 \leq \text{height}[i] \leq 10^4$

Input 1:

Input 2:

5 2 5 4 8 3 7	4 1 1 1 1
---------------	--------------

Output:

Output:

Max Water: 49	Max Water: 3
---------------	--------------

Question 4: Finding the Insertion Point for a New Calibration Value (Binary Search)

A lab keeps a sorted array of calibration reference values. Whenever a new value arrives, Ibrahim needs to find the exact index where it should be inserted to keep the array sorted — without scanning the whole array.

He uses a binary search variant that finds the first index whose value is greater than or equal to the new value:

1. Initialize $\text{low} = 0$, $\text{high} = N$.
 2. While $\text{low} < \text{high}$: compute $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$.
 3. If $\text{arr}[\text{mid}] < \text{target}$, set $\text{low} = \text{mid} + 1$. Otherwise, set $\text{high} = \text{mid}$.
 4. When the loop ends, low is the correct insertion index.
- The first line of input consists of two space-separated integers N and target .
 - The second line consists of N space-separated integers in non-decreasing order.
 - Print a single integer: the index at which target should be inserted, in the format: Insert At: X.
 - $1 \leq N \leq 10000$
 - $0 \leq \text{arr}[i], \text{target} \leq 10^4$

Input 1:

Input 2:

6 29 10 20 30 40 50 60	4 5 1 2 3 4
---------------------------	----------------

Output:

Output:

Insert At: 2	Insert At: 4
--------------	--------------

CATEGORY: HARD

Question 5: Locating a Packet in a Rotated Log Buffer (Modified Binary Search)

A circular network buffer stores log entries sorted by timestamp, but because it wraps around, the array as read out is a rotated sorted array (e.g., the oldest entries appear after the newest due to the wraparound point). Sana needs to locate a specific timestamp T in $O(\log N)$ time — a plain binary search won't work directly since the array isn't fully sorted end-to-end.

She adapts binary search to handle the rotation:

1. Initialize $low = 0$, $high = N - 1$.
 2. Compute mid. If $arr[mid] == target$, return mid.
 3. Determine which half (low to mid, or mid to high) is properly sorted by comparing $arr[low]$ and $arr[mid]$.
 4. Check if target lies within the value range of the sorted half. If it does, recurse/iterate into that half; otherwise, search the other half.
 5. Repeat until found or the range is empty.
- The first line of input consists of two space-separated integers N and target.
 - The second line consists of N space-separated distinct integers representing the rotated sorted array.
 - Print a single integer: the 0-indexed position of target in the array, in the format: Found At: X . If not present, print Found At: -1 .
 - $1 \leq N \leq 10000$
 - $0 \leq 10^5 \leq arr[i]$, target $\leq 10^5$
 - The array is a sorted array rotated at some unknown pivot; all elements are distinct.

Input 1:

Input 2:

7 0 4 5 6 7 0 1 2	5 3 4 5 6 7 8
----------------------	------------------

Output:

Output:

Found At: 4	Found At: -1
-------------	--------------

VIVA QUESTIONS:

1. "In binary search, why must the array be sorted before we can safely discard half the search space at each step? What breaks if it isn't?"
2. "Contrast the worst-case and best-case time complexity of linear search. Under what real-world circumstance would linear search actually outperform binary search on the same data?"
3. "In the rotated sorted array search problem, why can't we simply check if $\text{arr}[\text{mid}] < \text{target}$ the way we would in a standard binary search? What extra information do we need at each step?"
4. "The two-pointer merge technique for two sorted arrays runs in $O(N+M)$ time. Why would sorting the concatenation of both arrays from scratch be a worse approach, even though both are technically valid?"
5. "In the 'maximum rainwater between towers' problem, why is it always safe to move the pointer at the shorter tower inward, and never the taller one?"
6. " $\text{low} + (\text{high} - \text{low}) / 2$ is preferred over $(\text{low} + \text{high}) / 2$ for computing the midpoint in binary search. What specific failure does this guard against, and under what conditions would that failure actually occur in C?"
7. "If you were asked to implement binary search recursively instead of iteratively in C, what trade-off would you be making in terms of memory, and why does this matter for embedded systems with limited stack size?"
8. "How would you modify binary search to find the last occurrence of a repeated value in a sorted array, rather than just any occurrence?"
9. "Explain why the two-pointer technique for merging arrays cannot be applied directly if one of the arrays is unsorted. What would need to change?"
10. "In terms of cache behavior, why might linear search on a small array sometimes run faster in practice than binary search, despite having worse asymptotic complexity?"